Helios 解码器精读：解码步骤详解（论文 ↔ 代码对照）

> *面向学习者的精读笔记：把论文 FPGA-based Distributed Union-Find Decoder for Surface Codes（IEEE TQE 2024，下称"论文"）与仓库 `Helios_scalable_QEC-main/` 的 RTL 代码逐层对照，讲清楚 Helios 从拿到综合征到输出纠错（correction）的每一步。*
>
> *名词解释见同目录 `snowflake-on-helios-design.md` 的 §2 术语表；Snowflake 改进方案见该文档其余部分。*

目录

1. 阅读方法：三层结构
2. 解码器概览：输入是什么、输出是什么
3. 第 0 层：UF 解码器算法（论文 Algorithm 1）
4. 第 1 层：分布式化（论文 §III，Algorithm 2–6）
5. 第 2 层：RTL 实现（代码主线：单 FPGA 流程）
6. 端到端时间线：一个完整解码周期的阶段流转
7. 逐拍示例 1：四缺陷簇（论文 Fig.5 的完整推演）
8. 逐拍示例 2：单缺陷推向边界
9. 纠错输出与逻辑翻转判定
10. 论文 ↔ 代码对照总表
11. 两条代码线的差异（scalable 版 vs 单 FPGA 版）
12. 代码细节与"坑"（精读要点）
13. 动手建议与自测问题

---

1. 阅读方法：三层结构

论文和代码都是"三层"结构，精读时始终问自己"这一层在干什么、为什么能这么简化"：

层	论文	代码	核心问题
算法层	§II.B，Algorithm 1	无（理论）	UF 解码器为什么正确、为什么快
分布式算法层	§III，Algorithm 2–6	两个版本共有的逻辑	每个顶点只靠邻居信息，如何无中心协调地完成"并簇 + 奇偶统计"
硬件实现层	§IV–V，Algorithm 7/8 + Helios 架构	`design/` 下具体模块	全局同步、共享内存 S1–S4、单拍生长、合并/检查融合、剥离纠错

代码有两条线（详见 §11）：

- **单 FPGA 流程（推荐先精读这条）**：`Helios_single_FPGA_core.v` → `unified_controller`（`control_node_single_FPGA.v`）+ `single_FPGA_decoding_graph_dynamic_rsc.sv` → `processing_unit`（`processing_unit_single_FPGA_v2.v`）+ `neighbor_link`（`neighbor_link_v2.v`）。阶段名 `STAGE_GROW/MERGE/PEELING` 在 `parameters/parameters.sv` 中有定义，与论文 Algorithm 7/8 一一对应，仓库自带测试数据可跑。
- **可扩展（多 FPGA）流程**：`processing_unit.sv`（阶段 `STAGE_SPREAD_CLUSTER/GROW_BOUNDARY/SYNC_IS_ODD_CLUSTER`）+ `decoder_stage_controller.sv` + direct channel + hub/leaf 树。注意：这些阶段常量在仓库中缺定义（由 `generate.py` 生成产物补全），直接仿真需要先修复构建。

下文主流程按单 FPGA 线讲解，§11 单独对比两条线。

---

2. 解码器概览

输入：一个综合征（syndrome）——d 轮测量里，每个辅助比特"本轮结果 ≠ 上轮结果"即为缺陷（defect）。硬件上是一次性载入 `PU_COUNT = CODE_DISTANCE_X × CODE_DISTANCE_Z × MEASUREMENT_ROUNDS` 个比特（`measurements`），其中 MEASUREMENT_ROUNDS = max(X, Z)。

输出：一个纠错（correction）——一组被翻转的图边（`CORRECTION_NS/EW/UD`），保证"每个缺陷都被配对消除（或推向边界）"。硬件上按字节流串行吐出（§9）。

性能定位（论文 §VI）：d=21 唯象噪声平均 11.5ns/轮（100MHz），d=17 电路级噪声 23.7ns/轮；平均时延随 d **亚线性**增长（分布式 UF 用 $O(d^3)$ 量级并行资源换来的）。逻辑上等价于 Delfosse–Nickerson 的 UF 解码器，输出逐综合征一致（论文 §VI-A）。

关键洞察（论文 §III 开头）：UF 的 Growing 阶段每个顶点独立工作（只需要自己簇的奇偶性），Merging 阶段每个顶点只需要看自己的邻居——天然可分布。

---

3. 第 0 层：UF 解码器算法

论文 Algorithm 1 的循环（也是硬件状态机的外层循环）：

```
初始化：每个缺陷顶点 v 建一个单点簇（odd）
while 存在奇簇:
    Growing：对每个奇簇 C 的边界边 e：
             若 e.growth < e.w（边权）：e.growth += 1
             若 e.growth == e.w：e 加入 F（全生长边集合）
    Merging：对 F 中每条边 UNION 两端所在的簇
每个偶簇内建生成树 → 生成纠错（翻转树中配对路径上的边）
```

直觉：缺陷像"水渍"一样向外生长，两个水渍相遇（全生长边连接）就合并；合并让缺陷计数相消（奇数+奇数=偶数）。偶簇不再生长，奇簇继续长。长到触边界的簇可以把缺陷"推出"边界（等效配对到虚拟边界节点）。UF 的正确性保证：任意 s < d/2 个错误必可纠正。

每轮生长 +1、边权 w=2 的细节：论文用边权表示错误概率，唯象噪声下基本边权为 2，所以"生长两次长满一条边"。

---

4. 第 1 层：分布式化

论文 §III 把算法映射成"每顶点一个 PE + 一个控制器"：

4.1 每顶点状态（Algorithm 2 的变量，代码里一一对应）

论文状态	含义	单 FPGA 代码信号（processing_unit_single_FPGA_v2.v）
v:id	顶点唯一 ID（静态）	参数 ADDRESS（三维坐标编码）
v:m	本顶点是否缺陷	m（measurement 载入）
v:cid	所属簇 ID = 簇内最小 id	root
v:odd	本簇是否奇簇	odd
v:parent	指向父节点（生成树）	parent_vector（6 位 one-hot：指向哪个邻居端口）
v:st_odd	以 v 为根的子树缺陷奇偶	cluster_parity
v:stage / v:busy	当前阶段 / 是否有未完成工作	stage(来自 global_stage) / busy
e:growth	边生长值（0→w）	neighbor_link_v2.v 里的 growth 计数器

为什么 cid = 最小 id 就能实现 UNION？ 合并规则只有一条：*沿全生长边看到比自己小的 root 就采纳，并把该邻居设为自己 parent*。两个簇被全生长边连上后，两个方向的顶点都会在 1–2 个合并周期内向小 id 收敛，**不需要中心仲裁。**parent 关系顺带构成每簇的生成树（树的根 = cid 顶点），供奇偶汇聚用。

4.2 共享内存 S1–S4（论文 §III-C / §IV）

内存	内容	谁写	谁读
S1	cid, odd, parent, st_odd	自己	自己 + 相邻 PE
S2	codd, stage, busy	自己	自己 + 控制器
S3	e:growth（存于低 id 端点）	边的两个端点	两端
S4	global_stage	控制器	所有 PE

FPGA 实现就是寄存器（reg），相邻 PE 直接接线互相读（S1 的邻居可见性 = `neighbor_link` 的组合交叉线）；控制器对 busy/odd 做全局 OR（单 FPGA 版在 `unified_controller` 里 `busy <= |busy_PE`，多 FPGA 版是控制节点树）。

4.3 控制器（Algorithm 6）

```
循环:
  global_stage = growing;  等所有 PE 进入 growing 且 busy=0
  重复:
```

```
    global_stage = merging; 等所有 PE 进入 merging 且 busy=0
    global_stage = checking;等所有 PE 进入 checking
  直到 所有 PE 的 codd=false（无奇簇）
```

合并/检查要循环多次的原因：cid 洪泛、奇偶收敛、odd 回传都是"每拍传播一跳"，大簇需要多拍才稳定。

4.4 复杂度（论文 §III-F / §V-A）

- 分布算法最坏 $O(d^4 \log d)$：迭代数 ≤ d，每次迭代的合并/检查 ≤ $O(d^3)$ 拍（洪泛+收敛），控制器切换阶段 $O(\log d)$。
- FPGA 版去掉 log d 因子（阶段切换 1 拍）：最坏 **$3d^4 + 2d$** 拍（合并 = cid 广播 + st_odd 收敛 + odd 广播，各 ≤ d^3 拍）。
- 实际平均极快：d=13 时 10^6 次实验 97% 在 2 次迭代内完成（<250ns），最长也仅 920ns——大簇指数罕见。

---

5. 第 2 层：RTL 实现

5.1 模块层级（单 FPGA 流程）

```
Helios_single_FPGA_core.v (Helios_single_FPGA)
├── unified_controller  (control_node_single_FPGA.v)   ← 全局阶段机 + 字节流输入/输出协议
└── single_FPGA_decoding_graph_dynamic_rsc.sv           ← PE 阵列 + 全部边链路
    ├── processing_unit × PU_COUNT  (processing_unit_single_FPGA_v2.v)
    └── neighbor_link × 边数        (neighbor_link_v2.v)
```

顶层还有 `Helios_single_FPGA_wrapper.v`（随机错误发生器 `rand_gen_top`、BRAM 统计接口、`arm_communicator`），用于自测。

5.2 PE 阵列与地址

- 阵列：`i ∈ [0, CODE_DISTANCE_X)`，`j ∈ [0, CODE_DISTANCE_Z)`，`k ∈ [0, MEASUREMENT_ROUNDS)`（`single_FPGA_decoding_graph_dynamic_rsc.sv` 的 `pu_k/pu_i/pu_j` 生成循环）。
- 地址编码：`ADDRESS(i,j,k) = (k << (X_BIT+Z_BIT)) + (i << Z_BIT) + j` ——**k 是高位**，所以 k 越大的层 id 越大；簇根（最小 id）位于 k 小的一侧。k=0 层带有 UD 虚拟边界（未来边界，见 §5.4）。
- 邻居端口编号：`NORTH=0, SOUTH=1, WEST=2, EAST=3, DOWN=4, UP=5`（`single_FPGA_decoding_graph.v` 的宏）。

5.3 边链路 `neighbor_link_v2.v`

每条边一个实例，含：

- `growth` 计数器（宽度 `$clog2(WEIGHT+1)`，上限 WEIGHT）：`growth <= MIN(growth + a_increase + b_increase, WEIGHT)` ——**两端各贡献 +1**，所以两个奇簇之间的边一拍 +2、直接长满（论文 Fig.6 的原子 compare-and-update 优化）。
- `fully_grown = (growth >= WEIGHT)`。
- `BOUNDARY_CONDITION` 三种接线（见 §5.4）：
  - `0` 普通边：两端都可增；is_boundary=0；
  - `1` 边界边：只有 PE 端可增（`growth += a_increase`）；** `is_boundary = fully_grown` **（长满即"簇碰到虚拟边界节点"）；
  - `2` 不存在的边：fully_grown=0，占位。
- 信号交叉（组合逻辑，无拍延迟）：`a_root_out = b_root_in`、`a_parent_vector_out = b_parent_vector_in`、odd/parity/touching/peeling 系列信号同理双向直连——这就是 S1 共享内存的物理实现。
- `is_error`：剥离合出的纠错位（哪条边要翻转），由两端 PE 的 `neighbor_is_error` OR 得到。

5.4 三类边与边界（`single_FPGA_decoding_graph.v` 的生成循环）

边类型	数量（整图）	WEIGHT	边界条件
NS（南北，i 方向）	(X−1)·Z·U	WEIGHT_X	i=0 与 i=X 处为边界(1)，其余普通(0)
EW（东西，j 方向）	X·(Z+1)·U	WEIGHT_Z	j=0 与 j=Z 处为边界(1)
UD（上下，k 方向，时间轴）	X·Z·U	WEIGHT_M（k=0 边界边）/ WEIGHT_Z（内部）	**k=0 处为边界(1)**（未来边界），k=U 处不存在(2)

注意：边界边是"半个边"——只有 PE 端，另一端是虚拟边界节点。`is_boundary = fully_grown` 让虚拟边界节点的出现时机完全由生长进度决定。**两个空间边界 + 一个未来边界**，正是 forward 方法滑动窗口的解码图结构（Helios 论文 §VI-D 的滑动窗口扩展就是把 U 维拉长到 2d）。

5.5 PE 内部四个阶段（`processing_unit_single_FPGA_v2.v` 逐段解读）

PE 没有自己的状态机—— `stage <= global_stage`，所有 PE 全局同步，同一拍执行同一阶段（论文 §V-A 的核心简化）。

① 加载（STAGE_MEASUREMENT_LOADING，1 拍）

```
m <= measurement;            // 本顶点是否缺陷
root <= ADDRESS;             // 自己成簇，cid = 自己的 id
parent_vector <= 0;          // 无父
cluster_parity <= measurement; // 子树奇偶 = 自己
odd <= measurement;          // 单点簇：缺陷即奇簇
odd_to_children <= (measurement ? 6'h3f : 0);  // 向所有邻居广播自己的 odd
```

② 生长（STAGE_GROW，1 拍）

```
assign neighbor_increase = odd && (stage == STAGE_GROW) && (last_stage != STAGE_GROW);
```

奇簇的每个 PE 在 GROW 的第一拍**只发一次脉冲**（`last_stage != STAGE_GROW` 去重），所有入射边的 growth 各 +1。偶数簇不发脉冲 → 停止生长。

③ 合并（STAGE_MERGE，循环多拍直到全部 busy=0）——融合了论文的 merging + checking：

步骤 A：并簇（cid 洪泛）

```
min_val_less_8x_with_index( .values(neighbor_root),
                            .valids(neighbor_fully_grown & ~neighbor_is_boundary), ...)
// 若有效邻居中有比自己小的 root：
//    root <= 最小 root；parent_vector <= 对应端口 one-hot
```

组合逻辑树（`tree_compare_solver.sv` 里的 `min_val_*` 系列）在一拍内选出"全生长且非边界"邻居中的最小 root。边界边不参与（虚拟节点没有 root）。cid 每拍洪泛一跳：新合并的簇内，最小 id 从合并点向四周扩散。

步骤 B：子树奇偶汇聚（convergecast）

```
next_cluster_parity = m ^ (^(neighbor_parent_vector & child_cluster_parity));
next_cluster_touching_boundary = (|(neighbor_parent_vector & child_touching_boundary))
                               | (|neighbor_is_boundary);
```

"指向我的邻居"= 我的孩子（`neighbor_parent_vector & ...` 提取）。子树奇偶 = 自己 XOR 所有孩子的子树奇偶，每拍向根推进一层；触界标志同样沿树向上 OR。

步骤 C：odd 判定 + 回传（broadcast）

```
if (|parent_vector)      // 非根：odd = 父的 odd
    odd <= |(parent_vector & parent_odd);
else                     // 根：odd = 簇奇偶 && !触界
    odd <= next_cluster_parity & !next_cluster_touching_boundary;
odd_to_children <= (odd ? 6'h3f : 0);   // 向所有邻居广播
```

这是 UF 的核心规则：奇簇继续长，除非它已经碰到边界（触界的奇簇可以把缺陷推出边界，等效于"已配对"，不再生长）。根知道全簇奇偶后把 odd 沿树广播下去（`parent_odd` 线 = 父的 `odd_to_children`）。

步骤 D：busy 判定（checking 融合进来）

```
busy <= 1  若（root 将要改变）||（cluster_parity 改变）||（touching 改变）
           ||（非根：父 odd 与我的 odd 不一致）||（根：新 odd 与旧 odd 不一致）
       否则 0
```

任何 PE 有变化就继续留在 MERGE，全阵列安静（busy=0）才算合并完成——这就是把论文的 Checking 阶段融合进 Merging 的实现（Algorithm 7/8 的做法）。

④ 剥离（STAGE_PEELING，循环直到 busy=0）——生成纠错（论文 Algorithm 1 最后一行"build correction"）：

- `peeling_parity_completed`：根自举为 1，其余 = 父的值 → 沿树自上而下传播"开始剥离"信号；
- `peeling_complete`：所有孩子完成 && 自己的奇偶链完成；
- `peeling_m`：剥离奇偶 = `m ^ XOR(孩子们的 peeling_m) ^ odd`——每个节点把"子树里还没配对的缺陷数"吸收上来；
- **翻边判定**：
  - 内部边：`neighbor_is_error_internal = parent_vector & child_peeling_m`——孩子的子树里剩奇数个未配对缺陷，则"我–孩子"这条边进入纠错（配对路径穿过这条边）；
  - 边界边：若根/某节点 odd=1 且自身有边界边，`neighbor_is_error_border` 指向该边界边——剩下的单缺陷推向虚拟边界。
- `neighbor_link.is_error` 收集两端标记，输出到 `CORRECTION_*`。

5.6 控制器 `unified_controller`（`control_node_single_FPGA.v`）

状态机（阶段常量见 `parameters.sv`）：

```
IDLE --(收到 START_DECODING_MSG=0x01)--> PARAMETERS_LOADING --> IDLE
IDLE --(收到 MEASUREMENT_DATA_HEADER=0x02)--> MEASUREMENT_PREPARING
      (逐字节把测量数据移入 measurements 移位寄存器，攒满一轮)
      --> MEASUREMENT_LOADING (1 拍，注入本轮)
      --> 回到 PREPARING，直到 U 轮全部载入
      --> GROW (1 拍)
      --> MERGE ⇄ GROW          (delay ≥ MAXIMUM_DELAY 且 busy=0 时：
                                 odd_clusters=1 → 回 GROW；=0 → PEELING)
      --> PEELING               (delay ≥ MAXIMUM_DELAY 且 busy=0 → RESULT_VALID)
      --> RESULT_VALID × U 轮（逐轮输出纠错）--> IDLE
```

两个值得注意的时序细节：

1. **busy/odd 的一拍流水线**：`busy <= |busy_PE`、`odd_clusters <= |odd_clusters_PE` 是寄存器，FSM 读的是上一拍的值——控制器对阵列状态的反应天然滞后 1 拍。
2. **MAXIMUM_DELAY**：进入 MERGE/PEELING 后必须等 delay 计数满（单 FPGA 核用 3，控制节点默认 2）才检查 busy——给 PE 的 busy 计算留出稳定时间，避免误判。

---

6. 端到端时间线

```
拍序（相对）            全局阶段            PE 阵列在做什么
──────────────────────────────────────────────────────────────────────────────
载入阶段（每轮 1 拍）   MEASUREMENT_LOADING  每个 PE 记录 m，自成单点簇
L+1                     GROW                奇簇 PE 发 increase 脉冲，入射边 growth+1
M1..Mn（变长）          MERGE               每拍：cid 洪泛一跳 / 子树奇偶上推一层 / odd 回传一层
                                             任何变化 → busy=1 → 控制器留在 MERGE
（若 odd_clusters=1 → 回 GROW 再来一轮生长；否则 → PEELING）
P1..Pm（变长）          PEELING             "开始剥离"信号自上而下；剥离奇偶与翻边标记自下而上
R（×U 轮）               RESULT_VALID        correction 经 FIFO + serializer 逐字节输出
```

为什么 MERGE 要循环：合并是"洪泛 + 收敛 + 广播"三件事，都只能每拍传播一跳（组合逻辑跨 PE 的话关键路径会随 d 增长，破坏 f_max）。大簇直径 ~d^3 量级时循环拍数多，但平均簇极小，所以平均很快。

---

7. 逐拍示例 1

四缺陷簇（论文 Fig.5 的完整推演，与 RTL 逐拍核对）

拓扑：4 个缺陷 PE，id = 1,2,3,4（同层相邻，为便于讲解用一维编号）；存在的边：1–3、1–4、2–4，边权 WEIGHT=2。缺陷：全部（m=1）。

约定：**寄存器在拍结束时更新，下一拍起新值可见**；"children" = 指向我的邻居（parent_vector）。

7.1 生长 + 合并（论文 Fig.5 的 8 拍 = 1 生长 + 7 合并）

拍	阶段	发生了什么（变化用 粗体）

L	加载	全部：root=id，parity=1，odd=1，m=1，parent=∅
G1	GROW	1,3,4,2 都 odd → 各边两端各 +1 → 边(1,3)(1,4)(2,4) growth=2 → **fully grown**
M1	MERGE	PE3 见邻居 1（root 1<3）→ **root=1, parent→1**；PE4 见邻居 1,2 → **root=1, parent→1**；PE1 见 3,4（root 3,4，≥自己）不动；PE2 见 4（root 4>2）不动。busy: 3,4
M2	MERGE	PE2 见 PE4 现在 root=1<2 → **root=1, parent→4**；PE1 的孩子=3,4：parity=1^1^1=1（不变）；PE3/4：odd=父(1)的 odd=1。busy: 2
M3	MERGE	PE4 的孩子=2：parity=1^1=0 → **parity=0**（子树{4,2}偶）。busy: 4
M4	MERGE	PE1 见孩子 4 的 parity=0：parity=1^1^0=0 → **parity=0**；根 odd=0&!0=**0**。busy: 1
M5	MERGE	PE3/PE4 见父(1)的 odd=0 → **odd=0**。busy: 3,4
M6	MERGE	PE2 见父(4)的 odd=0 → **odd=0**。busy: 2
M7	MERGE	全阵列无变化 → **busy=0**。控制器：odd=0 → 进 PEELING

与论文 Fig.5 描述完全一致：(b) 一拍长满 3 条边 → (c) 3、4 归 1 → (d) 2 归 4、同时 1 算子树奇偶 → (e)(f) 子树 4、1 的 st_odd 依次更新、根得 odd=0 → (g) odd 两拍传遍全簇 → (h) 第 8 拍无变化，控制器换阶段。

7.2 剥离（6 拍）与最终纠错

拍	pc（peeling_parity_completed）变化	pcom（complete）变化	peeling_m 计算
P1	根 PE1: **1**	无	全员先存 m
P2	PE3, PE4: 得父值 **1**	无	—
P3	PE2: 得父(4)值 **1**	**PE3 完成**（无孩子）	PE3: 1^0^0=**1**
P4	—	**PE2 完成**	PE2: 1^0^0=**1**
P5	—	**PE4 完成**（孩子2已完）	PE4: 1^child2(1)^0=**0**
P6	—	**PE1 完成**	PE1: 1^child3(1)^child4(0)^0=**0**

翻边标记：PE1 的孩子 3 的 peeling_m=1 → **边(1,3) 进纠错**；PE4 的孩子 2 的 peeling_m=1 → **边(2,4) 进纠错。**

最终 correction = {(1,3), (2,4)}：4 个缺陷被恰好配成 (1,3) 与 (2,4) 两对，每条纠错路径各翻转一条边。✓

（注：实际控制器在 M7 后还要等 MAXIMUM_DELAY 拍才进 PEELING，不影响阵列内时序；P1–P6 之后同理。）

这个例子展示了全部核心机制：单拍长满（两端 +2）、最小 id 做根、cid 每拍一跳、奇偶沿生成树收敛、odd 沿树回传、busy 自组织终止、树剥离出最小配对。

---

8. 逐拍示例 2

单缺陷推向边界：一行 3 个 PE（id 0,1,2），西边界接 PE0、东边界接 PE2（边界边 WEIGHT=2），缺陷只在 PE1（m=0,1,0）。

拍	阶段	关键变化
G1	GROW	PE1 发脉冲 → 边(0,1)(1,2) growth=1
M1	MERGE	无全生长边 → 无变化；busy=0，odd=1 → 回 GROW
G2	GROW	PE1 再脉冲 → 两内部边 growth=2 **fully grown**
M2	MERGE	PE1 见邻居 0（root 0<1）→ **root=0, parent→西**；PE2 见邻居 1 → **root=1, parent→西**
M3	MERGE	PE0 的孩子=1：parity=0^1=**1**，根 odd=**1**（簇{0,1}变奇）；PE1 的 odd=父(0)的 0 → **0**；PE2 不变
M4	MERGE	PE1：odd=父(0)的 1 → **1**；PE2：odd=父(1)的 0 → **0**
M5	MERGE	PE2：odd=父(1)的 1 → **1**。全簇{0,1,2} odd=1，parity=1 ✓。回 GROW
G3	GROW	三个 PE 都 odd → 内部边已长满（饱和）；**PE0 的西边界边 +1**，PE2 的东边界边 +1

M6	MERGE	无新全生长内部边 → 无变化。回 GROW
G4	GROW	**两条边界边 growth=2 → fully grown → is_boundary=1**
M7	MERGE	PE0：touching=**1**；根 odd = parity(1)&!1 = **0**（触界即"已配对"！）
M8	MERGE	PE1：odd=父(0)的 0 → **0**
M9	MERGE	PE2：odd=父(1)的 0 → **0**
M10	MERGE	无变化 → busy=0；odd=0 → **PEELING**

剥离：PE0 是根，簇奇偶=1（奇数个缺陷）且触界 → 根保留 odd=1 → **西边界边进纠错**；内部：PE1 的 peeling_m=1 → **边(0,1) 进纠错**；PE2 的 peeling_m=0。

最终 correction = {西边界边, 边(0,1)} = 缺陷 PE1 → PE0 → 西边界的完整路径。✓（缺陷被"推出"西边界，这正是 UF 的边界配对行为。）

这个例子展示：奇簇继续扩张（第二、三、四轮生长）、簇沿时间/空间合并、**触界奇簇停止生长的规则**（odd = parity & !touching）、以及剥离时"单缺陷沿树走到边界"的路径生成。

---

9. 纠错输出与逻辑翻转判定

9.1 correction 的编码

`CORRECTION_NS / EW / UD` 各按 (i,j,k) 编号展开成一维位向量（`single_FPGA_decoding_graph.v` 的 `CORR_INDEX_*` 宏）。**每轮**的纠错位数为 NS + EW + UD（dynamic_rsc 版直接输出每轮切片）。位=1 表示该边被翻转。

9.2 输出字节协议（`unified_controller`）

RESULT_VALID 阶段把纠错经 `fifo_wrapper` → `serializer`（8 位窄口）逐字节吐出，前面附 3 个字节头：

```
byte0 = iteration_counter        （解码迭代了几轮生长）
byte1,2 = cycle_counter 高/低   （总拍数，评测用）
然后 = 本轮 correction 字节流，逐轮重复 U 次
```

9.3 逻辑翻转（logical flip）判定

解码纠错后，如果"被翻转的边"里存在一条**从西边界到东边界**的路径，逻辑比特翻转。scalable 版有 `get_boundry_cardinality_3d.sv`（当前被 stub 成常数 0，注释里有完整实现）：统计每个簇是否触界、奇偶如何，输出 final_cardinality。**学习时可以把它当作"正确性检查点"**：一个触界的奇簇贡献一次边界配对；奇数次西边界配对 ⇒ 逻辑翻转。

9.4 边界平局规则（一个隐藏细节）

剥离阶段 `casex (neighbor_is_boundary)` 按端口顺序 N,S,W,E,D,U 找第一个触界端口——**西先于东。**这给"同时触两界的奇簇"提供了确定性配对方向（与 Snowflake 设计文档 §5.1 讨论的"西向优先"完全对应，Helios 代码里其实已经隐含了这条规则）。

---

10. 论文 ↔ 代码对照总表

论文	单 FPGA 代码	说明
Algorithm 1（UF 主循环 + 最后建纠错）	`unified_controller` 的 GROW/MERGE 循环 + `STAGE_PEELING`	外层 while = MERGE⇄GROW 循环
Algorithm 2 L26–L27（初始化）	PE 的 `STAGE_MEASUREMENT_LOADING` 分支	单点簇初始化
Algorithm 3（Growing，原子 compare-and-update）	`neighbor_increase` 脉冲 + `neighbor_link` 的 `growth <= MIN(growth+a+b, WEIGHT)`	双端 +2 = 论文 Fig.6 的优化；**代码无 cid 检查**（见 §12.2）
Algorithm 4（Merging：采小 cid、设 parent）	`min_val_less_8x_with_index` + root/parent 更新	valids = fully_grown & ~is_boundary
Algorithm 4 L62–65（st_odd / odd）	`next_cluster_parity` / `odd` 三段式更新	根算奇偶，非根抄父

Algorithm 5（Checking）	MERGE 阶段的 `busy` 计算	checking 融合进 merging（Algorithm 7 的做法）
Algorithm 6（控制器）	`unified_controller` 状态机	busy/odd 经寄存器（1 拍流水线）
Algorithm 7/8（FPGA 定制版）	全套：生长 1 拍、merge+check 融合、terminate 检测 = odd_clusters=0	一一对应
Algorithm 1 L25（簇内建生成树纠错）	`STAGE_PEELING` + `neighbor_is_error`	树剥离
§IV S1–S4 共享内存	寄存器 + `neighbor_link` 组合交叉线 + 控制器 OR	单 FPGA 版简化了 S4
§IV 控制节点树	scalable 版 `control_node_child/root_FPGA.v`	单 FPGA 版退化为一个 `unified_controller`
§V-B 上下文切换 Helios-n	`single_FPGA_decoding_graph_dynamic_rsc.sv`（rsc = resource）	PE 状态换入换出，见 §12.4

---

11. 两条代码线的差异

维度	**scalable 版（`processing_unit.sv` 等）**	**单 FPGA 版（`processing_unit_single_FPGA_v2.v` 等）**
阶段	SPREAD_CLUSTER（并簇）→ GROW_BOUNDARY（生长+边界代价）→ SYNC_IS_ODD_CLUSTER（奇簇同步）	GROW → MERGE（含检查）→ PEELING
奇偶汇聚	**direct channel 消息**：谁并了簇就发 `[新根地址, odd, touching]` 消息，经 3D 路由（`tree_distance_3d_solver`）多跳投递给新根	**生成树收敛**：parent 树 + 逐拍 XOR 上推
边界模型	每 PE 三个 cost 计数器（`boundary_cost_x/z/ud` + `boundary_increased_*`），生长耗尽代价即触界	边界边 `BOUNDARY_CONDITION=1`，`is_boundary = fully_grown`
控制器	`decoder_stage_controller.sv` + 控制节点树（hub/leaf）	`unified_controller`（单节点 OR）
纠错输出	`get_boundry_cardinality_3d`（stub）	完整 PEELING 剥离
多 FPGA	hub/leaf + routing table + serdes + 互连模型	无
与论文关系	对应 §III 分布式算法（但奇偶走消息而非收敛）	**与 §V Algorithm 7/8 一一对应**

学习建议：**先彻底吃透单 FPGA 线**（它与论文算法描述最贴、且仓库测试数据齐全），再把 scalable 线当作"另一种奇偶机制 + 多 FPGA 通信"的变体来对比阅读。

---

12. 代码细节与"坑"

1. **生长脉冲去重**：`neighbor_increase = odd && (stage==STAGE_GROW) && (last_stage != STAGE_GROW)` ——没有 `last_stage` 判断的话，同一 GROW 阶段若持续多拍会重复生长（控制器保证 GROW 只 1 拍，这是双保险）。
2. **生长缺 cid 检查**：论文 Algorithm 3 L45 有 `u:cid != v:cid`（只长簇边界边），RTL 两个版本都没有。影响：同一奇簇内部、尚未长满的边也会被两端各 +1。由于簇内边几乎都已长满（否则不会同簇），且 +2 被 MIN 饱和，实际影响极小——但这是"代码与论文不完全一致"的典型例子，值得自己构造图（含环）验证。
3. **busy 的一拍流水线**：控制器看到的 busy/odd 是上一拍寄存器值；加上 MAXIMUM_DELAY（2 或 3），换阶段比"阵列真正安静"晚 2–4 拍——评测 cycle_counter 时要把这些常数额外拍算进去（论文的 $3d^4+2d$ 公式里 "+2" 就是这类开销）。
4. **rsc 版（resource-shared）**：`Helios_single_FPGA_core.v` 实际用的是 `single_FPGA_decoding_graph_dynamic_rsc.sv`，与论文 §V-B 的 Helios-n 上下文切换对应（虚拟 PE 映射到物理 PE）。测量数据经 `measurement/measurement_out` 链逐轮下移注入（新轮从 k 最大端进入）。
5. **构建问题**：scalable 线的阶段常量（`STAGE_SPREAD_CLUSTER` 等）在仓库中无定义，`design/generated/` 需由 `build_scripts/generate.py` 生成（依赖 numpy/pymetis）；单 FPGA 线用的常量在 `parameters/parameters.sv` 中有定义，可直接仿真（`scripts/total.tcl` 建工程，`test_benches/` 里有 d=3~13 的输入/期望输出）。

6. **边界平局的端口顺序**（§9.4）：剥离的 `casex` 隐含西/北优先——理解这一点对后面做 Snowflake 的"西向优先"改造有用。
7. **动态权重**：`WEIGHT_X/Z/M` 是独立参数，`neighbor_link` 的计数器宽度 `$clog2(WEIGHT+1)` 随之变化——这是论文 §VI-D 加权边扩展的落点（权重刻度直接改 growth 上限）。

---

13. 动手建议与自测问题

建议的动手顺序：

1. 用 `scripts/total.tcl` 建 Vivado 仿真工程，跑 `test_benches/full_tests/single_FPGA_verification_test.sv`（d=3 起步），对照 `test_data/input_data_3.txt / output_data_3.txt`；
2. 波形上盯 4 个信号：`global_stage`、某 PE 的 `root/odd/cluster_parity/busy`、某条 `neighbor_link` 的 `growth/fully_grown`；
3. 把 §7 的四缺陷用例灌进去，逐拍对表；
4. 再读 scalable 线，对比两种奇偶机制。

自测问题（能答上就说明吃透了）：

1. 为什么 cid 取最小 id 就能完成并查集的 UNION，而无需任何中心协调？根和 parent 树是怎么自然形成的？
2. 为什么"触界的奇簇"要停止生长？写出代码里对应的一行。
3. MERGE 阶段为什么必须循环多拍？哪些信号每拍只传播一跳？
4. busy 信号与论文 Algorithm 5 的 checking 是什么关系？busy=0 在全阵列意味着什么？
5. 剥离阶段为什么能保证"每个偶簇恰好配对、每个触界奇簇恰好把一条路径推向边界"？
6. 若把 GROW 的 `last_stage != STAGE_GROW` 条件删掉会发生什么？
7. 论文 §V-A 说 FPGA 版把复杂度从 $O(d^4 \log d)$ 降到 $O(d^4)$，log d 因子是从哪里省掉的？
8. k=0 层的 UD 边界边在解码图中代表什么？为什么它的权重单独用 WEIGHT_M？

关联文档：`snowflake-on-helios-design.md`（术语表 + 基于本文档的 Snowflake 改进方案）。